

Lecture 07: Recursion

Today Lecture's Contents

- Recursion
- Recursion Functions
- Base Condition
- Work through Examples of Recursion

Recursion

- Recursion is repetition (by self-reference)
 - it is caused when a function calls/invokes itself.
 - Such a process will repeat forever unless terminated by some control structure.

Recursion

- So far, we have learned about control structures (for loop, while, do while etc.) that allow C++ to iterate a set of statements a number of times.
- In addition to iteration, C++ can repeat an action by having a function call itself.
 - This is called recursion. In some cases it is more suitable than iteration.

Recursion

- Let's look at a very simple example;
 - in this case we can see that by using recursion we can make some difficult problems very trivial...
 - many of these problems would be very difficult to solve if you only were able to use iteration.
 - *trace through the following problem...showing how the stack frame works*

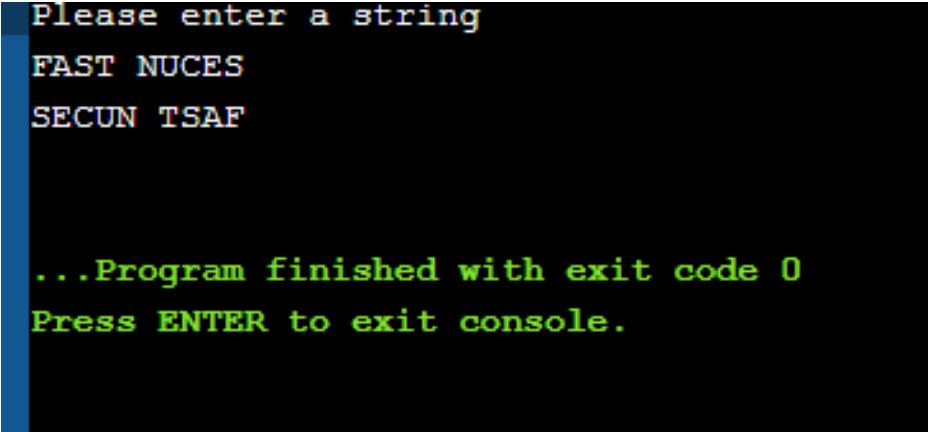
Example 1: Recursion

- What is the purpose of the following?

```
void strange(void);           //function prototype

int main() {
    cout <<"Please enter a string" <<endl;
        strange();
        cout <<endl;
        return 0;
}

void strange(void) {
    char ch;
    cin.get(ch);
    if (!cin.eof() && ch != '\n') { //Base Condition
        strange();
        cout <<ch;
    }
}
```



```
Please enter a string
FAST NUCES
SECUN TSAF

...Program finished with exit code 0
Press ENTER to exit console.
```

Recursion

- This program writes the reverse of what was entered at the keyboard, no matter how many characters were entered!
 - Try to write an equally simple program just using the iterative statements we know about;
 - it would be difficult to make it behave the same without limiting the number of characters that can be entered or using up a lot of memory with a huge array of characters!
 - Notice, with recursion, we didn't have to even use an array!!

Recursion—Stopping criteria

- In order for your recursive calls to be useful, they must be designed so that your program will ultimately terminate.
- As with iteration or looping, there is danger of creating a recursive function that is an infinite loop!
- We need to be careful to prevent infinite repetition.
- Therefore, when designing a recursive function
 - one of the first steps should be to determine what the stopping condition should be

Recursion—Stopping criteria

- The best way to do this is to use
 - an if statement to determine if a recursive call should be made -- depending on the value of some conditional expression.
- Eventually, every recursive set of calls should reach a point that does not require recursion (i.e., this will stop recursion).
 - That point is called 'base condition'

```
void strange(void) {  
    char ch;  
    cin.get(ch);  
    if (!cin.eof() && ch != '\n') { //Base Condition  
        strange();  
        cout <<ch;  
    }  
}
```

Recursion--Requirements

- Therefore, there are three requirements when using recursion:
- Every recursive function must contain a control structure that prevents further recursion when a certain state is reached.
- That state must be able to be reached each time you run the program.
- When that state is reached, the function must have completed its computation and (if the function returns a value) return the appropriate value for each recursive call. *don't forget to have the function "use" the returned value...if there is one!*
 - *Example on next slide shows recursion function returns values*

Example 2: Solving Factorial

- walk through the following:

```
int factorial(int n)
{
    if (n < 2)
        return 1;
    else
        return (n * factorial(n-1));
}
```

Compare and contrast with the iterative version.
Which is better? Why?

factorial function

- Assume the number typed is 3, that is, numb=3.

fac(3) :

3 <= 1 ?

No.

fac(3) = 3 * fac(2)

fac(2) :

2 <= 1 ?

No.

fac(2) = 2 * fac(1)

fac(1) :

1 <= 1 ?

Yes.

return 1

fac(2) = 2 * 1 = 2

return fac(2)

fac(3) = 3 * 2 = 6

return fac(3)

fac(3) has the value 6

```
int fac(int numb) {  
    if(numb<=1)  
        return 1;  
    else  
        return numb * fac(numb-1);  
}
```

factorial function

For certain problems (such as the factorial function), a recursive solution often leads to short and elegant code. Compare the recursive solution with the iterative solution:

Recursive solution

```
int fac(int numb) {  
    if (numb<=1)  
        return 1;  
    else  
        return numb*fac (numb-1) ;  
}
```

Iterative solution

```
int fac(int numb) {  
    int product=1;  
    while (numb>1) {  
        product *= numb;  
        numb--;  
    }  
    return product;  
}
```

Recursion

We have to pay a price for recursion:

- calling a function **consumes more time and memory** than adjusting a loop counter.

In less demanding applications recursion is an attractive alternative for iteration (for the right problems!)

Making Mistake in using Iteration

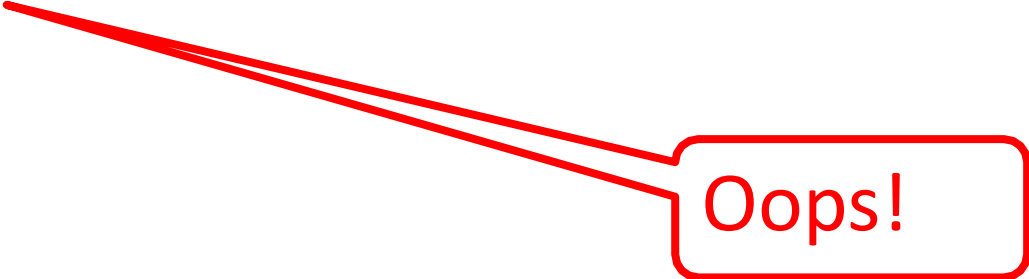
If we use iteration, we must be careful not to create an infinite loop by accident:

```
for(int incr=1; incr!=10;incr+=2)  
...
```

```
int result = 1;  
while(result >0){  
...  
result++;  
}
```



Oops!



Oops!

Making Mistake in using Recursion

Similarly, if we use recursion we must be careful not to create an infinite chain of function calls:

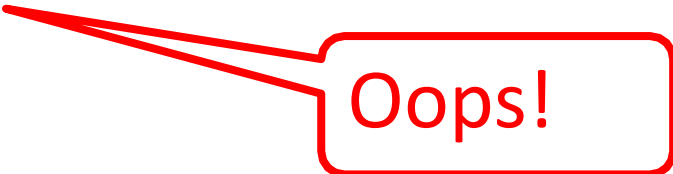
```
int fac(int numb) {  
    return numb * fac(numb-1);  
}
```

Or:

```
int fac(int numb) {  
    if (numb<=1)  
        return 1;  
    else  
        return numb * fac(numb+1);  
}
```



Oops!
No termination
condition



Oops!

Example 3: Fibonacci Numbers

- Fibonacci numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

where each number is the sum of the preceding two.

- Recursive definition:

- $F(0) = 0;$

- $F(1) = 1;$

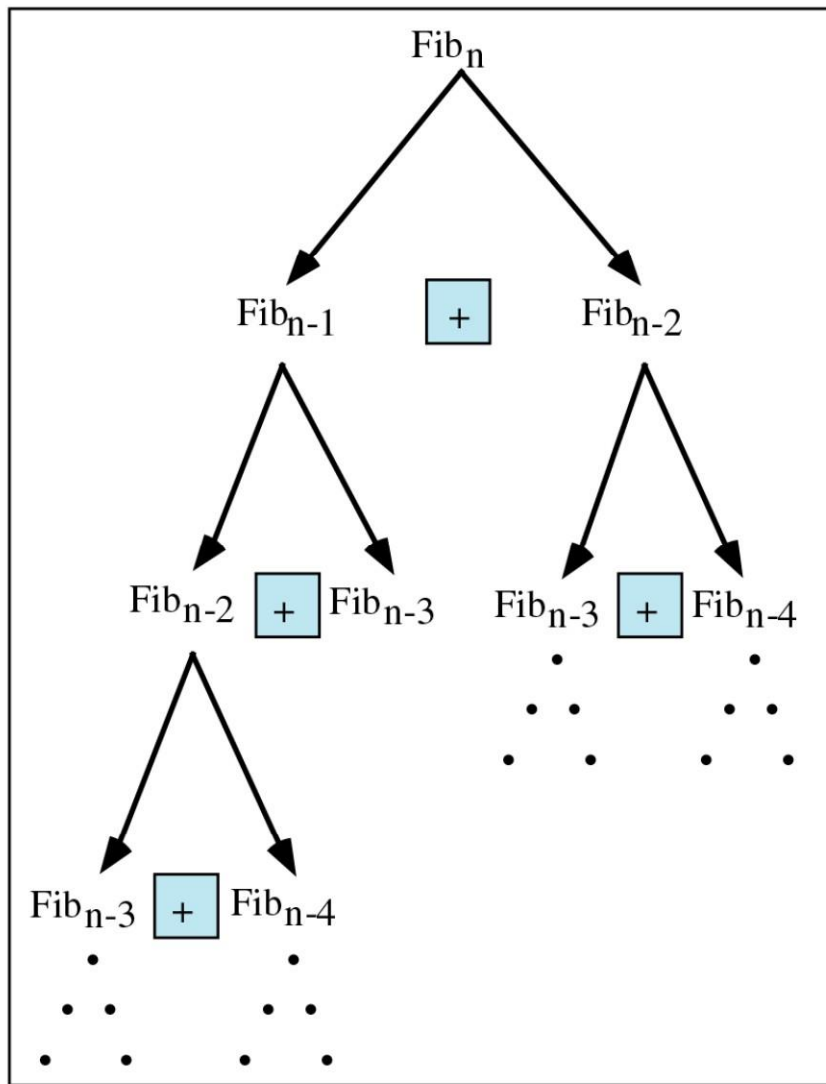
- $F(\text{number}) = F(\text{number}-1) + F(\text{number}-2);$

Example 3: Fibonacci numbers

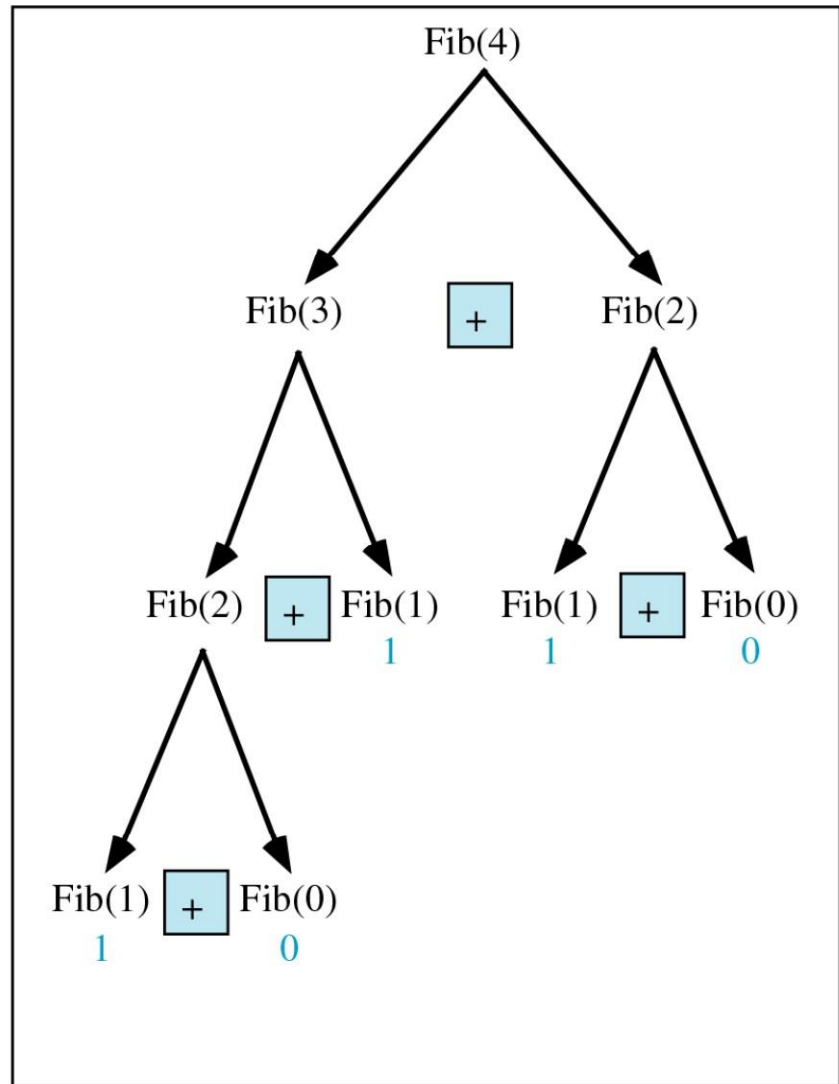
//Calculate Fibonacci numbers using recursive function.

```
int fib(int number)
{
    if (number == 0) return 0;
    if (number == 1) return 1;
    return (fib(number-1) + fib(number-2));
}
```

```
int main(){    // driver function
    int inp_number;
    cout << "Please enter an integer: ";
    cin >> inp_number;
    cout << "The Fibonacci number for "<< inp_number
        << " is "<< fib(inp_number)<<endl;
    return 0;
}
```



(a) $\text{Fib}(n)$



(b) $\text{Fib}(4)$

Trace a Fibonacci Number

- Assume the input number is 4, that is, num=4:

fib(4) :

4 == 0 ? No; 4 == 1? No.

fib(4) = fib(3) + fib(2)

fib(3) :

3 == 0 ? No; 3 == 1? No.

fib(3) = fib(2) + fib(1)

fib(2) :

2 == 0? No; 2==1? No.

fib(2) = fib(1)+fib(0)

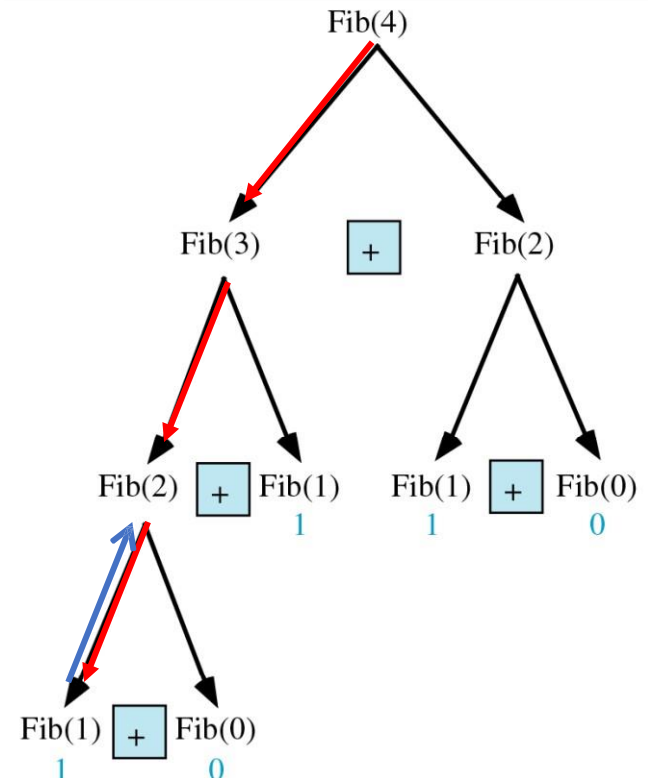
fib(1) :

1== 0 ? No; 1 == 1? Yes.

fib(1) = 1;

return fib(1);

```
int fib(int num)
{
    if (num == 0) return 0;
    if (num == 1) return 1;
    return
        (fib(num-1)+fib(num-2));
}
```

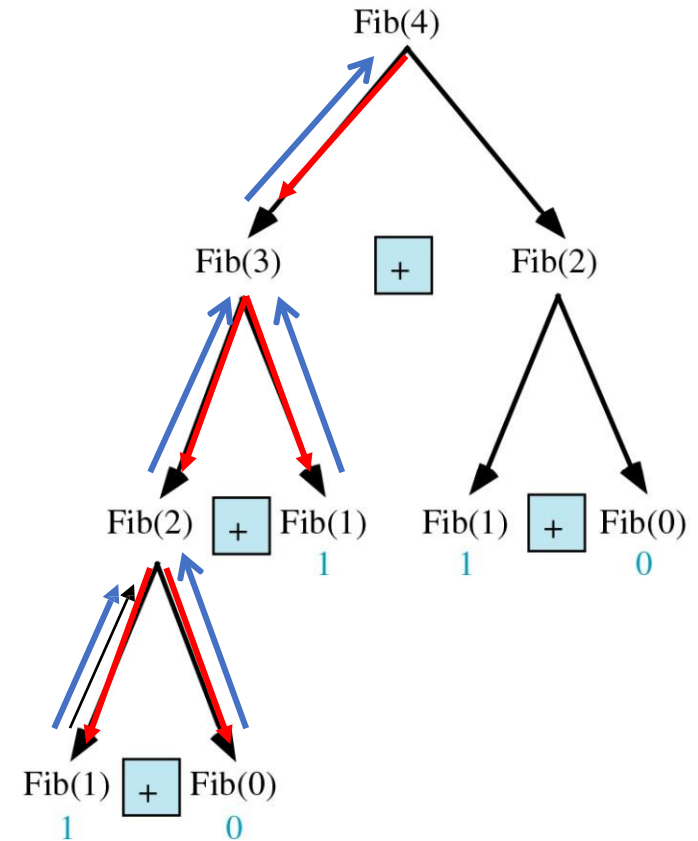


Trace a Fibonacci Number

```
fib(0):  
    0 == 0 ? Yes.  
    fib(0) = 0;  
    return fib(0);
```

```
fib(2) = 1 + 0 = 1;  
return fib(2);  
fib(3) = 1 + fib(1)
```

```
fib(1):  
    1 == 0 ? No; 1 == 1? Yes  
    fib(1) = 1;  
    return fib(1);  
fib(3) = 1 + 1 = 2;  
return fib(3)
```



Homework

- Tower of Hanoi through recursion

Obeying the following rules:

1. Only one disk can be transfer at a time.
2. Each move consists of taking the upper disk from one of the peg and placing it on the top of another peg i.e. a disk can only be moved if it is the uppermost disk of the peg.
3. Never a larger disk is placed on a smaller disk during the transfer.

